

INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE MONTERREY

Construye | MLflow Project y tracking avanzado | Colaborativo

Equipo 03

Anaí Lael Barrera Magaña — A01797380

Gerardo González Martínez — A01840096

Miguel Ángel Ramírez Castillo — A01840045

Sebastián Estrada García — A00805402

Operaciones de aprendizaje automático

25 de octubre de 2026

Repositorio: https://github.com/mna-mlops-tc5061-03/TC5061-Proyecto-Equipo03

Integrante	Matrícula	Aportación principal en esta entrega (issues)
Miguel Ángel Ramírez Castillo	A01840045	Candidatos de P1, P2 y P3 (#40 a #42) e integración continua con GitHub Actions
Gerardo González Martínez	A01840096	Entorno y entradas del MLproject, validación temporal y métrica de selección, un archivo por pregunta, etiquetas y artefactos, búsqueda con nested runs y README (#35 a #39, #44 y #51)
Anaí Lael Barrera Magaña	A01797380	Servidor de tracking con SQLite (#33) y revisión de la búsqueda, los candidatos y el README
Sebastián Estrada García	A00805402	Versión de datos por contenido (#34), Model Registry y `make registry` (#43), corrida oficial (#45), capturas y este reporte (#46 y #47)

Introducción

En la primera entrega dejamos una base reproducible para tres preguntas de la operación de MiBici, el sistema de bicicleta pública del Área Metropolitana de Guadalajara: cuántos viajes saldrán de cada estación al día siguiente (P1, regresión), qué estaciones estarán en riesgo de quedarse sin bicicletas o sin anclajes en el siguiente bloque de tres horas (P2, clasificación) y qué grupos de estaciones comparten patrones de uso (P3, clustering). Cada pregunta tenía un baseline y un candidato registrados en el experimento `mibici-equipo03`.

En esta segunda entrega convertimos el repositorio en un MLflow Project que se ejecuta con un solo comando, comparamos de forma ordenada tres candidatos por pregunta y una búsqueda de hiperparámetros contra su baseline, guardamos en cada corrida los artefactos que respaldan su resultado y llevamos el mejor modelo de cada pregunta al Model Registry, donde el alias `champion` marca la versión en producción. El reporte sigue el orden de los pasos de la actividad.

1. Proyecto reproducible

1.1 Servidor de tracking con base de datos

El Model Registry necesita que MLflow guarde sus metadatos en una base de datos, no en carpetas. MLflow admite como almacén de metadatos cualquier base compatible con SQLAlchemy, como SQLite (MLflow, s.f.-d). Usamos un servidor local con SQLite y una carpeta para los artefactos:

```
mlflow server --backend-store-uri sqlite:///mlflow.db \
  --artifacts-destination ./mlartifacts --port 5001
```

Usamos el puerto 5001 porque en macOS el 5000 lo ocupa AirPlay. La URI del servidor (`MLFLOW_TRACKING_URI=http://127.0.0.1:5001`) se define en un archivo `.env`, que se crea a par-

tir de `.env.example` y no se versiona. `mlflow.db`, `mlruns/` y `mlartifacts/` están en `.gitignore`, y el comando está documentado en el README.

1.2 El archivo MLproject

El MLproject declara el nombre del proyecto, su entorno y sus puntos de entrada, que son las tres piezas de un MLflow Project (MLflow, s.f.-c). El entorno es `python_env.yaml`, con Python 3.13 y las mismas versiones fijas de `requirements.txt`. La Tabla 1 resume los puntos de entrada.

Entrada	Qué hace	Parámetros (valor por defecto)
`ingest`	Descarga, limpia y versiona los datos de MiBici (`mi_bici/dataset.py`)	—
`train`	Entrena y registra un modelo del catálogo de una pregunta	`question` (`p1`), `model` (`baseline`), `params` (vacío: los hiperparámetros del catálogo)
`search`	Búsqueda de hiperparámetros con un run padre y un run hijo por combinación	`question` (`p1`), `model` (`hgb`), `grid` (vacío: la cuadrícula del catálogo)

Tabla 1. *Puntos de entrada del MLproject.*

Los valores por defecto no repiten rutas, fechas ni semillas. El MLproject solo dice qué pregunta y qué modelo correr, y todo lo demás (periodo, cortes, semilla, umbrales y experimento) sale de `mi_bici/config.py`. Así hay un solo lugar donde cambiar la configuración, y ningún valor puede quedar distinto entre el MLproject y el código. Los hiperparámetros entran como un solo parámetro `params` con el formato `"clave=valor,..."`, porque cada modelo del catálogo tiene los suyos. Los que no se indican toman el valor del catálogo.

1.3 Comandos para reproducir los resultados

Desde un clon limpio, con Python 3.13:

```
git clone https://github.com/mna-mlops-tc5061-03/TC5061-Proyecto-Equipo03.git
cd TC5061-Proyecto-Equipo03
python3.13 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
mlflow server --backend-store-uri sqlite:///mlflow.db \
  --artifacts-destination ./mlartifacts --port 5001        # en otra terminal
set -a && source .env && set +a
mlflow run . -e ingest --env-manager local --experiment-name mibici-equipo03
make experimentos   # baseline, candidatos y búsqueda de P1, P2 y P3 (18 corridas)
make registry       # champion y challenger de cada pregunta
```

`make experimentos` lanza con `mlflow run` las 18 corridas de la entrega: por pregunta, el baseline, los tres candidatos del catálogo, la búsqueda con nested runs y, como candidato, la mejor combinación de esa búsqueda, con los valores que dejó en la etiqueta `mejor_params`. Para reproducir solo el modelo elegido de cada pregunta:

```
mlflow run . -e train -P question=p1 -P model=hgb \
  -P params="learning_rate=0.1,max_leaf_nodes=63" \
  --env-manager local --experiment-name mibici-equipo03
mlflow run . -e train -P question=p2 -P model=hgb \
  -P params="learning_rate=0.05,max_leaf_nodes=127" \
  --env-manager local --experiment-name mibici-equipo03
mlflow run . -e train -P question=p3 -P model=kmeans \
  -P params="n_clusters=3" \
  --env-manager local --experiment-name mibici-equipo03
```

- `--env-manager local` usa el entorno virtual ya creado con `requirements.txt`, que tiene las mismas versiones que `python_env.yaml`. Sin esa opción, MLflow crea un entorno aislado a partir de `python_env.yaml` y necesita `pyenv` o `uv` (MLflow, s.f.-c).
- `--experiment-name` es obligatorio porque el código fija el experimento `mibici-equipo03` y MLflow rechaza un run creado en otro.
- La URI del servidor se carga del `.env` porque `mlflow run` le pasa al código la de la terminal desde la que se lanza. Sin ella, los runs irían a un `mlflow.db` local en lugar del servidor. `make experimentos` la carga por su cuenta y se detiene si no hay `.env`.

Corrida oficial. Se hizo en un clon nuevo de `main` (commit `ee51bc9`) con una `mlflow.db` nueva: la ingesta con `mlflow run` (51 s, con la descarga de los 12 meses del portal), `make experimentos` (3.5 min), `make registry` y el cambio de alias de P3 que se explica en la sección 6. `git status` quedó limpio antes y después. Los 36 runs de entrenamiento y búsqueda (las 18 corridas y los 18 hijos de las búsquedas) tienen `git_commit = ee51bc9`, `git_dirty = False` y `data_version = 4ab2182e7493`.

1.4 Qué hace reproducible cada corrida

Treveil et al. (2020) enumeran lo que hay que documentar para reproducir un modelo: supuestos, aleatoriedad, datos, configuración, resultados, implementación y entorno (pp. 57–58). Además, la reproducibilidad completa exige versionar todo lo que interviene, incluidos los datos de entrenamiento y evaluación y un registro del entorno de software (p. 26). La Tabla 2 muestra cómo cubre cada punto el proyecto. Como resume Lauchande (2021), `mlflow run` ejecuta el proyecto según la receta del MLproject y con los parámetros indicados, lo que permite reproducir una corrida por completo (p. 19).

Condición	Cómo se cubre en el proyecto
Supuestos	Umbral de riesgo, bloque de horas, periodo y cortes en `config.py`, registrados como parámetros del run
Aleatoriedad	Semilla fija (`SEMILLA = 42`) que se pasa a cada estimador y se registra como parámetro
Datos	`data_version`: hash del manifiesto de la ingesta, que guarda un hash del contenido de cada tabla. El manifiesto va como artefacto y la tabla de entrenamiento agregada con `mlflow.log_input`
Configuración	Parámetros del run y la configuración resuelta en `config/`
Resultados	Métricas de validación y prueba con el mismo nombre en todas las corridas, y gráficas en `plots/`
Implementación	Etiquetas `git_commit` y `git_dirty=False`: el commit exacto basta para volver a correr el código
Entorno	`python_env.yaml` y `requirements.txt` con versiones fijas

Tabla 2. *Cómo queda registrada cada condición de reproducibilidad (Treveil et al., 2020, pp. 57–58).*

La versión de los datos, igual en cualquier computadora. En la primera entrega, `data_version` era el hash de un manifiesto que guardaba el MD5 de los bytes de cada archivo Parquet. Con los mismos datos, el MD5 de la tabla de estaciones salía distinto en dos computadoras del equipo, así que había que restaurar el manifiesto a mano después de la ingesta. En esta entrega (#34, PR #55) el manifiesto guarda un hash del contenido de cada tabla: se calcula un hash por fila, los hashes se ordenan para que el orden de las filas no importe y se combinan en un SHA-256.

Al probarlo en las dos computadoras apareció la causa real. El lector rápido de decimales de `pandas.read_csv` no redondea de forma exacta, y en algunas coordenadas de 17 dígitos del catálogo de estaciones el último dígito binario cambiaba según la computadora (en una, 12 latitudes y 30 longitudes). Leer ese CSV con redondeo exacto (`float_precision="round_trip"`) lo resolvió. Es un ejemplo de lo que advierten Treveil et al. (2020): una implementación ligeramente distinta, o una versión distinta de un paquete, puede cambiar los resultados de formas difíciles de prever (pp. 57–58). En las tres ingestas de esta entrega, cada una en un clon nuevo con los datos recién descargados del portal, el manifiesto salió idéntico al del repositorio y `data_version` fue `4ab2182e7493`.

2. Diseño de la experimentación

2.1 Misma partición y mismas métricas

Conservamos la partición temporal y las métricas de la primera entrega para que la comparación contra el baseline sea válida. Lo único que agregamos es un conjunto de validación dentro del periodo de entrenamiento (Tabla 3).

Conjunto	Periodo	Para qué
Entrenamiento para validar	1 oct 2025 a 31 may 2026	Ajustar cada modelo antes de medirlo en validación
Validación	1 jun 2026 a 31 jul 2026	Comparar y elegir modelos
Prueba	1 ago 2026 a 30 sep 2026	Confirmar la mejora. Solo descarta candidatos que no le ganen al baseline, no ordena

Tabla 3. *Partición temporal definida en* `config.py`.

Cada corrida ajusta primero el modelo con los meses anteriores al corte de validación y lo mide en validación. Después lo vuelve a ajustar con todo el periodo de entrenamiento y lo mide en prueba. Así seguimos la recomendación de Treveil et al. (2020): usar como conjunto de evaluación los registros más recientes, que estiman mejor cómo se comportará el modelo con datos futuros, y usar el conjunto de prueba una sola vez, con la mejor combinación ya elegida (p. 52). En la primera entrega comparábamos directamente en prueba. Con un solo candidato por pregunta no había elección que sesgar, pero con varios candidatos y una búsqueda, elegir con la prueba la contaminaría. P3 no tiene etiqueta ni horizonte, así que se mide con el periodo de entrenamiento.

Las métricas son las mismas de la primera entrega, ahora con el conjunto en el nombre (`val_` o `test_`): MAE y, como apoyo, RMSE y R^2 en P1; average precision y, como apoyo, recall, precisión y F1 en P2; silueta y, como apoyo, Davies-Bouldin y Calinski-Harabasz en P3 (scikit-learn developers, s.f.-a, s.f.-b). Revisar varias métricas y no solo la que se optimiza ayuda a detectar comportamientos indeseados del modelo (Treveil et al., 2020, p. 53). Los baselines dan en prueba exactamente lo mismo que en la primera entrega (MAE de 6.907, average precision de 0.168 y silueta de 0.017), lo que confirma que la regla y la partición de prueba no cambiaron.

2.2 Candidatos por pregunta

Cada pregunta tiene su catálogo de modelos en su propio archivo (`p1_demanda.py`, `p2_riesgo.py` y `p3_grupos.py`). Un run es una pregunta y un modelo del catálogo con sus hiperparámetros. La Tabla 4 resume lo que se probó: tres familias distintas por pregunta, para comparar algoritmos y no solo hiperparámetros.

Pregunta	Baseline (regla, no se entrena)	Candidatos
P1 · Demanda	Mismo día de la semana anterior (`viajes_lag_7`)	`hgb` (HistGradientBoosting), `rf` (RandomForest, 100 árboles, profundidad 15) y `ridge` (regresión Ridge con variables escaladas)
P2 · Riesgo	Riesgo del mismo bloque la semana anterior	`hgb`, `rf` (100 árboles, profundidad 15) y `logreg` (regresión logística escalada), los tres con `class_weight="balanced"` y el umbral de decisión calibrado en validación
P3 · Grupos	Agrupar por municipio	`kmeans`, `birch` y `gmm` (mezcla gaussiana), con variables escaladas y *k* de 3 a 8 elegido por silueta

Tabla 4. *Baseline y candidatos de cada pregunta.*

Empezar con una regla simple como baseline permite saber si un modelo aporta algo. Treveil et al. (2020) recomiendan evaluar un modelo comparándolo con lo que existía antes, sea un modelo previo o un proceso basado en reglas (p. 51), y Lauchande (2021) también parte de un pipeline con una heurística simple antes de probar modelos (p. 62).

Umbral de decisión en P2 (#60). Los clasificadores de P2 se entrenan con `class_weight="balanced"`, que compensa la clase minoritaria pero infla las probabilidades de riesgo. Con el umbral de 0.5, en la primera entrega el candidato marcaba en riesgo muchos bloques (recall de 0.870 y precisión de 0.146). Ahora cada candidato elige en validación el umbral que maximiza el F1, y ese mismo umbral se aplica en prueba. El umbral queda entre 0.80 y 0.84 según el modelo y se registra como métrica (`val_umbral_decision`). La average precision no depende del umbral, que solo cambia el recall y la precisión con que se reporta la decisión de sí o no.

2.3 Búsqueda de hiperparámetros con nested runs

Organizamos cada búsqueda como un run padre con un run hijo por combinación, con `mlflow.start_run(nested=True)` (MLflow, s.f.-e). Así cada combinación queda registrada como una corrida propia, y la interfaz permite comparar los hijos y ver el efecto de cada hiperparámetro en la métrica (Lauchande, 2021, pp. 80–83). Hicimos una búsqueda por pregunta, sobre la familia con mejor resultado inicial (Tabla 5).

Los hijos solo se miden en validación, así que la prueba no participa en la búsqueda. El padre guarda la tabla de resultados (`data/busqueda.csv`) y la mejor combinación en la etiqueta `mejor_params`. Como `make registry` no toma los hijos de una búsqueda, la mejor combinación se vuelve a correr como candidato. Sus valores se copiaron de la etiqueta `mejor_params` de cada búsqueda al `Makefile`, así que `make experimentos` la corre con `-P params` fijo.

Búsqueda	Combinación	Métrica de validación
P1 · `hgb` (`val_mae`, menos es mejor)	`learning_rate=0.05`, `max_leaf_nodes` 31 / 63 / 127	5.890 / 5.861 / 5.849
	`learning_rate=0.1`, `max_leaf_nodes` 31 / **63** / 127	5.842 / **5.828** / 5.853
P2 · `hgb` (`val_average_precision`, más es mejor)	`learning_rate=0.05`, `max_leaf_nodes` 31 / 63 / **127**	0.3242 / 0.3284 / **0.3293**
	`learning_rate=0.1`, `max_leaf_nodes` 31 / 63 / 127	0.3205 / 0.3288 / 0.3257
P3 · `kmeans` (`train_silhouette`, más es mejor)	`n_clusters` **3** / 4 / 5 / 6 / 7 / 8	**0.096** / 0.056 / 0.081 / 0.060 / 0.088 / 0.051

Tabla 5. *Búsquedas con nested runs, con seis hijos cada una. En negritas, la mejor combinación.*

Las diferencias dentro de cada búsqueda son pequeñas. En P1, la mejor combinación mejora el `val_mae` del `hgb` por defecto en 0.014 viajes, y en P2 dos combinaciones quedan a menos de 0.001 de average precision. Treveil et al. (2020) advierten que probar todas las combinaciones se vuelve inmanejable y recomiendan fijar un presupuesto de tiempo o de cómputo y un umbral de aceptación (p. 50). Por eso las búsquedas son cuadrículas pequeñas y acotadas (la más cara, la de P2, tardó 25 s con sus seis combinaciones), y el umbral de aceptación es ganarle al baseline (sección 5).

Las Figuras 1 a 3 muestran la búsqueda de P2 en la interfaz: el padre con sus seis hijos junto al baseline y los candidatos, y dos formas de comparar a los hijos.

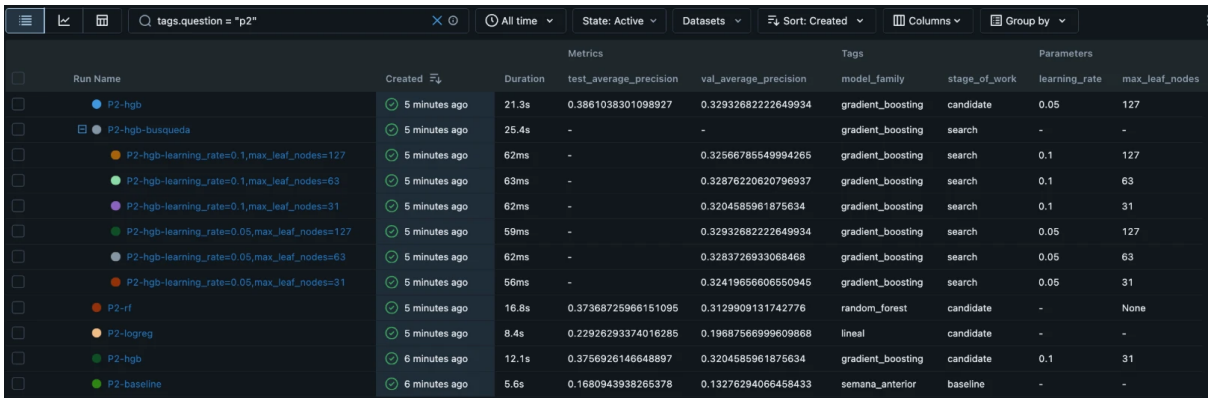


Figura 1. *Corridas de P2 en `mibici-equipo03`, filtradas con `tags.question = "p2"`: baseline, candidatos y la búsqueda `P2-hgb-busqueda` expandida con sus seis runs hijos (`stage_of_work = search`). Arriba, el candidato con la mejor combinación de la búsqueda.*



Figura 2. *Coordenadas paralelas de los seis hijos de la búsqueda de P2: `learning_rate` y `max_leaf_nodes` contra `val_average_precision`.*

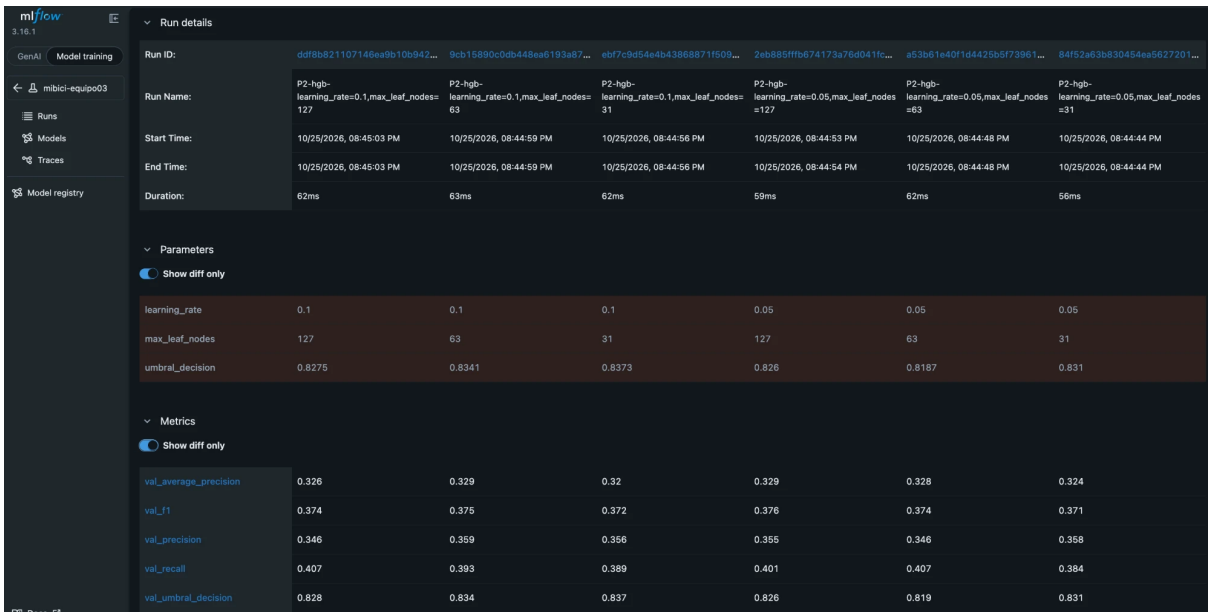


Figura 3. *Comparación de los seis hijos de la búsqueda de P2 con «Show diff only»: hiperparámetros, umbral de decisión calibrado y métricas de validación. A tres decimales, `0.1 / 63` y `0.05 / 127` empatan en 0.329. Con cuatro gana `0.05 / 127` (0.3293 contra 0.3288).*

2.4 Etiquetas para filtrar y comparar

Además de `equipo`, `git_commit` y `git_dirty`, cada corrida lleva `question` (`p1`, `p2` o `p3`), `model_family` y `stage_of_work` (`baseline`, `candidate` o `search`). Con ellas, en la interfaz se filtra, por ejemplo, `tags.question = "p2" and tags.stage_of_work = "candidate"` (Figura 1), y `make registry` usa las mismas etiquetas para encontrar el baseline y los candidatos de cada pregunta.

3. Resultados

Todas las cifras son de la corrida oficial: `mlflow.db` nueva, árbol limpio, commit `ee51bc9` de `main` y `data_version 4ab2182e7493`. La mejora se calcula sobre el baseline: en P1, cuánto baja el MAE, y en P2 y P3, cuánto sube la métrica. `entrenamiento_segundos` es el tiempo del ajuste

final con todo el periodo de entrenamiento, en segundos (columna «s»).

3.1 P1 · Demanda diaria por estación

Run	Hiperparámetros clave	MAE val	MAE prueba	RMSE prueba	R^2 prueba	Mejora val / prueba	s
P1-baseline	Semana anterior	7.835	6.907	12.993	0.858	—	0.00
P1-rf	100 árboles, profundidad 15	5.898	5.561	10.420	0.909	24.7 / 19.5 %	1.14
P1-ridge	alpha 1.0	6.664	6.433	11.308	0.892	14.9 / 6.9 %	0.00
P1-hgb	Tasa 0.1, 31 hojas	5.842	5.563	10.402	0.909	25.4 / 19.5 %	0.92
P1-hgb (mejor de la búsqueda)	Tasa 0.1, 63 hojas	**5.828**	**5.539**	**10.365**	**0.910**	25.6 / 19.8 %	1.01

Tabla 6. *P1 · Demanda diaria por estación (regresión). Se elige con `val_mae`, en viajes: menos es mejor. Tasa: `learning_rate`. Hojas: `max_leaf_nodes`. En negritas, el modelo elegido.*

Los cuatro candidatos le ganan al baseline en validación y en prueba. El mejor es `hgb` con la combinación de la búsqueda: baja el MAE de 7.835 a 5.828 viajes en validación (−25.6 %) y de 6.907 a 5.539 en prueba (−19.8 %). Frente a una demanda media de 29.8 viajes por estación y día en el periodo de prueba, el error promedio pasa del 23 % al 19 %. `rf` queda prácticamente empatado en prueba (5.561), pero su modelo pesa 84 MB contra menos de 1 MB de `hgb`. `ridge` es el más débil (6.433 en prueba), porque un modelo lineal no captura las interacciones entre rezagos, día y capacidad.

Todos los modelos, incluido el baseline, tienen menos error en prueba que en validación, así que la diferencia viene del periodo (agosto y septiembre son más predecibles que junio y julio) y no de sobreajuste. El RMSE y el R^2 confirman el resultado: el elegido tiene el menor RMSE de prueba (10.365) y el mayor R^2 (0.910).

3.2 P2 · Riesgo por estación y bloque de tres horas

Run	Hiperparámetros clave	AP val	AP prueba	Recall prueba	Precisión prueba	Mejora val / prueba	s
P2-baseline	Semana anterior	0.133	0.168	0.365	0.385	—	0.00
P2-hgb	Tasa 0.1, 31 hojas, umbral 0.837	0.320	0.376	0.415	0.396	141 / 124 %	1.98
P2-logreg	C 1.0, umbral 0.800	0.197	0.229	0.387	0.319	48 / 36 %	0.15
P2-rf	100 árboles, profundidad 15, umbral 0.806	0.313	0.374	0.411	0.385	136 / 122 %	4.69
P2-hgb (mejor de la búsqueda)	Tasa 0.05, 127 hojas, umbral 0.826	**0.329**	**0.386**	**0.425**	0.393	148 / 130 %	6.48

Tabla 7. *P2 · Riesgo por estación y bloque de 3 horas (clasificación). Se elige con `val_average_precision`: más es mejor. El recall y la precisión de prueba usan el umbral calibrado en validación. En negritas, el modelo elegido.*

Los bloques en riesgo son pocos: 4.82 % en entrenamiento, 3.75 % en validación y 4.36 % en prueba. Los tres candidatos, todos con `class_weight="balanced"`, le ganan al baseline en average precision en validación y en prueba. `hgb` con la combinación de la búsqueda es el mejor: 0.329 en validación y 0.386 en prueba, contra 0.133 y 0.168 del baseline (+148 % y +130 %). `rf` queda cerca (0.374 en prueba) y `logreg` lejos (0.229).

Con el umbral calibrado, el elegido tiene en prueba un recall de 0.425 y una precisión de 0.393, contra 0.365 y 0.385 del baseline. Es decir, detecta más bloques en riesgo sin perder precisión. Con el umbral de 0.5 de la primera entrega, el recall era alto (0.870), pero la precisión muy baja (0.146), lo que se traduce en demasiadas visitas innecesarias para el equipo de rebalanceo.

3.3 P3 · Grupos de estaciones

Run	Hiperparámetros clave	Silueta	Mejora	Davies-Bouldin	Calinski-Harabasz	Estaciones por grupo	s
P3-baseline	Municipio	0.017	—	5.141	5.0	329 / 82 / 15	0.00
P3-kmeans	*k* = 3, 10 inicios	0.096	452 %	2.552	39.9	179 / 186 / 61	0.10
P3-birch	*k* = 3, umbral 0.5	0.091	425 %	2.621	31.4	213 / 187 / 26	0.75
P3-gmm	3 componentes	0.277	1498 %	3.070	15.7	365 / 47 / 14	0.10
P3-kmeans (mejor de la búsqueda)	*k* = 3, 10 inicios	**0.096**	452 %	**2.552**	**39.9**	**179 / 186 / 61**	0.04

Tabla 8. *P3 · Grupos de estaciones (clustering, 426 estaciones). P3 no tiene prueba: todas las métricas son del periodo de entrenamiento. Se elige con `train_silhouette`: más es mejor. Davies-Bouldin: menos es mejor. Calinski-Harabasz: más es mejor. En negritas, el modelo elegido tras revisar la estabilidad (sección 5).*

Los tres candidatos le ganan al baseline en las tres métricas internas, y los tres eligen *k* = 3, el mínimo del rango. Por silueta, `gmm` sale muy arriba (0.277 contra 0.096 de `kmeans` y 0.091 de `birch`), pero Davies-Bouldin y Calinski-Harabasz prefieren a `kmeans` (2.552 y 39.9 contra 3.070 y 15.7). La razón está en el tamaño de los grupos: `gmm` deja 365 de las 426 estaciones en un solo grupo y separa dos grupos pequeños de 47 y 14, mientras que `kmeans` forma grupos de 179, 186 y 61. Con el 86 % de las estaciones en un solo grupo, esa silueta alta sirve de poco para planear.

Las siluetas de todos los modelos son bajas, lo que indica patrones de uso continuos más que grupos muy separados, igual que en la primera entrega. Los grupos de `kmeans` se describen con su perfil horario: uno con pico a las 8 h (179 estaciones), otro con pico a las 18 h y más viajes (186) y uno plano entre las 7 y las 13 h con menos viajes (61). La sección 5 explica la elección.

4. Registro de artefactos

Cada corrida guarda con MLflow lo que respalda su resultado, organizado en subcarpetas desde `ExperimentoBase.ejecutar()`, igual para las tres preguntas (Tabla 9).

Dónde	Contenido	Para qué
Modelo del run (`models:/m-...`)	Modelo con `log_model`, firma (`signature`, inferida con `infer_signature`) y ejemplo de entrada (`input_example.json`), serializado con skops	Cargarlo con la misma interfaz desde el Registry y saber qué columnas espera
`data/`	`manifest.json` de la ingesta. En P3, además, el grupo de cada estación, el resumen por grupo y la silueta por *k*	Saber con qué versión de los datos se entrenó
Entradas del run	La tabla de entrenamiento con `mlflow.log_input` (`p1_train`, `p2_train` y `p3_train`), contexto `training`: solo esquema y *digest*, sin filas	Identificar el conjunto exacto de entrenamiento
`plots/`	P1: real contra predicho. P2: matriz de confusión y curva precisión-recall. P3: mapa y perfil horario de los grupos	Revisar el desempeño más allá de una cifra
`config/`	`parametros.json` (parámetros resueltos), `config.py`, `MLproject`, `python_env.yaml` y `requirements.txt`	Volver a correrla con la misma configuración

Tabla 9. *Artefactos de una corrida candidata.*

En MLflow 3, el modelo no queda como carpeta dentro de los artefactos del run, sino como un modelo propio ligado al run, con su identificador `models:/m-...` (MLflow, s.f.-d). Por eso la firma se ve en la página del modelo (Figura 5) y no en la pestaña de artefactos del run (Figura 4). Los modelos se guardan con skops, un formato que no ejecuta código al cargarse. Solo `birch` se guarda con cloudpickle, porque skops no admite su estructura interna. Cada run registra además `entrenamiento_segundos` y, en el modelo, su tamaño, que usamos para comparar el costo.

`mlflow.log_input` registra la tabla con un *digest*, una huella que identifica el conjunto de datos, y con su origen. Eso da el linaje de los datos y permite comprobar que dos corridas usaron los mismos (MLflow, s.f.-a). El manifiesto completa esa información con las filas y el hash del contenido de cada tabla procesada.

Sin datos personales. La ingesta elimina los identificadores del viaje y del usuario. Los artefactos, la tabla de `log_input` y el ejemplo de entrada del modelo solo tienen variables agregadas por estación y periodo. El género y el año de nacimiento solo aparecen en P3 como proporciones o medianas por grupo de estaciones, para describir los perfiles, y no entran al modelo. Treveil et al. (2020) señalan que la gobernanza de datos debe asegurar que no se usen datos personales para entrenar un modelo y que anonimizar no siempre basta, porque a veces aún se puede identificar a una persona (p. 36). Por eso no guardamos datos por viaje en ningún artefacto.

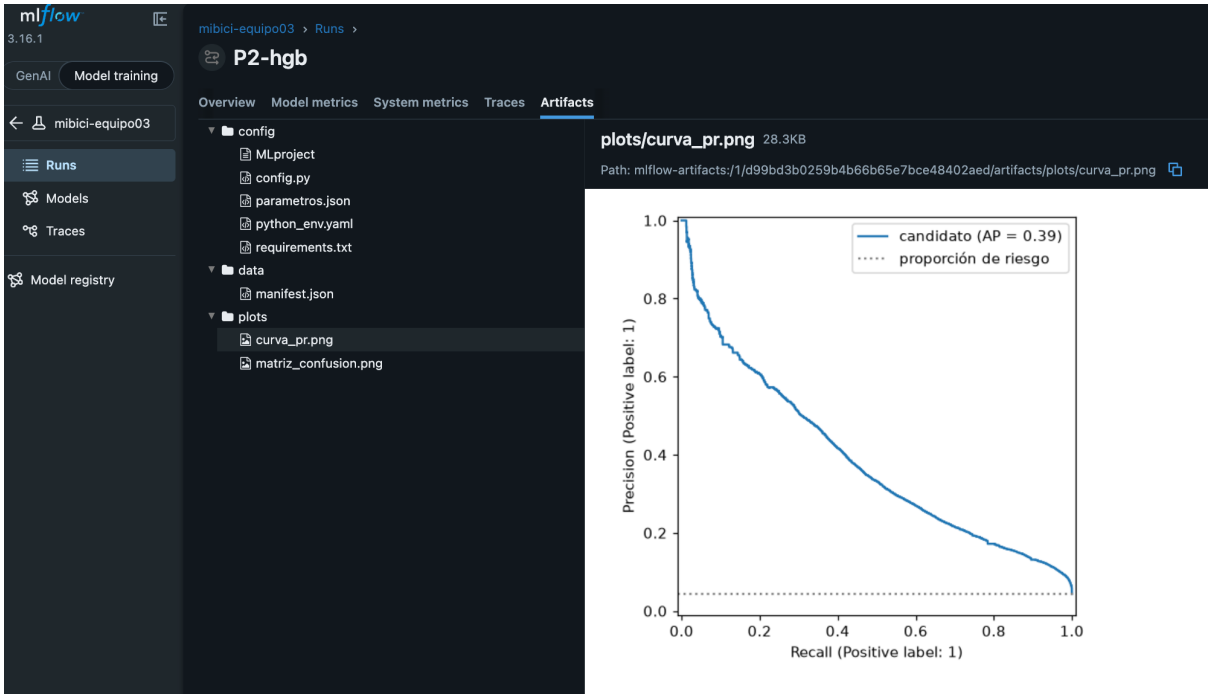


Figura 4. *Artefactos del champion de P2 (`P2-hgb` con `learning_rate=0.05` y `max_leaf_nodes=127`): `config/`, `data/manifest.json` y `plots/`, con la curva precisión-recall en prueba (AP = 0.39) frente a la proporción de bloques en riesgo.*

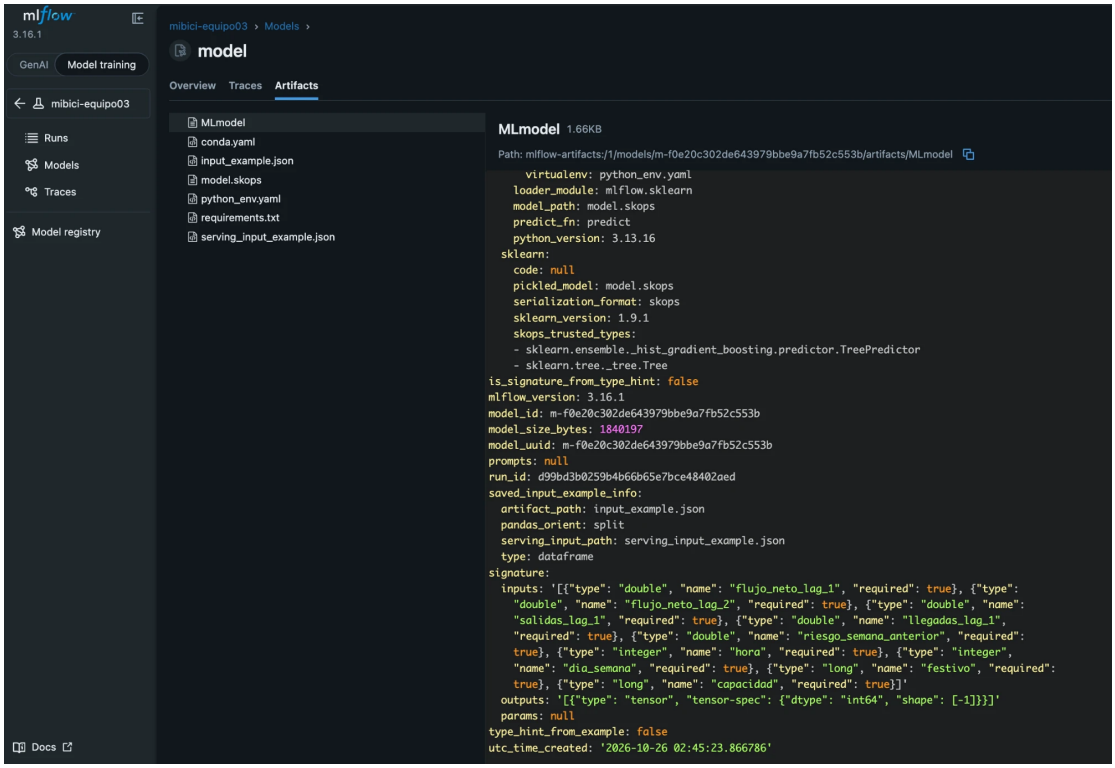


Figura 5. *Archivo `MLmodel` del mismo modelo: firma con las nueve variables de entrada y su tipo, salida de tipo entero, ejemplo de entrada, formato skops y run de origen.*

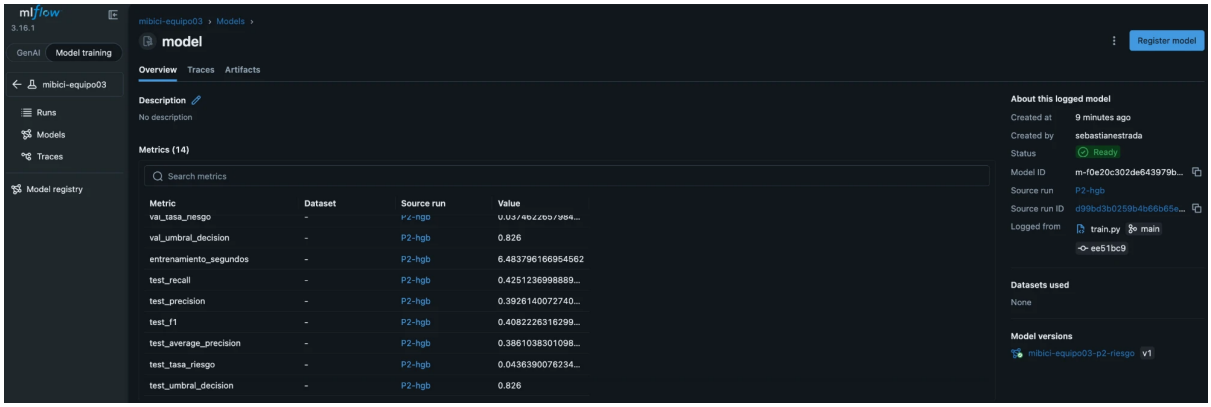


Figura 6. *Página del modelo del run: métricas, run de origen, rama y commit con que se entrenó y la versión registrada* `mibici-equipo03-p2-riesgo` *v1.*

5. Selección del mejor modelo

5.1 Criterios

Elegir el mejor modelo requiere criterios cuantitativos, como el error promedio, y cualitativos, como la facilidad de explicar el modelo o de desplegarlo (Treveil et al., 2020, p. 26). Además, la experimentación busca un equilibrio entre la mejora del modelo y su costo de cómputo, porque siempre se puede mejorar un poco más (p. 50). Con eso definimos cuatro criterios:

1. **Mejora sobre el baseline.** Un candidato solo es elegible si le gana al baseline en la métrica de selección en validación y también en prueba. Si gana en validación, pero pierde en prueba, se descarta como probable sobreajuste.
2. **Métrica ligada a la decisión de negocio.** Se elige con la métrica de la pregunta, no con la que más mejore.
3. **Estabilidad.** Se prefiere el modelo cuya mejora en validación se confirma en prueba y cuyo resultado no depende de la semilla.
4. **Costo.** Entre modelos con desempeño parecido, se prefiere el que entrena más rápido (`entrenamiento_segundos`), pesa menos y es más fácil de explicar.

Los criterios 1 y 2 están en `make registry` (`mi_bici/modeling/registry.py`): busca los candidatos de cada pregunta, descarta los que tienen cambios sin guardar o usan otros datos u otra partición que el baseline, conserva los que le ganan en validación y en prueba, y los ordena por la métrica de selección de validación. La prueba funciona solo como compuerta: descarta al candidato que no confirme su mejora, pero el orden lo da la validación. Los criterios 3 y 4 los revisa el equipo en las tablas de la sección 3 antes de aceptar el resultado. La Tabla 10 resume la decisión.

Pregunta	Modelo elegido	Mejora y métrica de negocio	Estabilidad	Costo
P1 · Demanda	`hgb` (0.1 / 63 hojas)	MAE −25.6 % en validación y −19.8 % en prueba: 5.5 viajes de error por estación y día, contra 6.9	La mejora se confirma en prueba	1.0 s y menos de 1 MB, contra 84 MB de `rf`
P2 · Riesgo	`hgb` (0.05 / 127 hojas)	Average precision +148 % en validación y +130 % en prueba: ordena mejor qué estaciones visitar	La mejora se confirma en prueba	6.5 s contra 2.0 s del `hgb` por defecto, todavía bajo
P3 · Grupos	`kmeans` (k = 3)	Silueta 0.096 contra 0.017 del municipio, con grupos que describen perfiles de uso	Silueta 0.098 ± 0.001 con las semillas 0 a 9, contra 0.264 ± 0.033 de `gmm`	Menos de 1 s, despreciable en los tres candidatos

Tabla 10. *Modelo elegido por pregunta y cómo cumple cada criterio.*

5.2 Modelo elegido por pregunta

P1 · Demanda. Elegimos `hgb` con tasa de aprendizaje de 0.1 y 63 hojas. Su MAE es de 5.828 en validación, 25.6 % mejor que el baseline, y de 5.539 en prueba, 19.8 % mejor. El MAE se lee en viajes, la misma unidad con que el operador reparte la flota. En promedio, el modelo se equivoca por 5.5 viajes por estación y día, contra 6.9 del baseline. Es estable, porque la mejora se confirma en prueba, y barato: entrena en 1.0 s y pesa menos de 1 MB. `rf` da casi el mismo error de prueba, pero pesa 84 MB, unas cien veces más, sin ganar exactitud. Por eso queda fuera aunque sea elegible.

P2 · Riesgo. Elegimos `hgb` con tasa de aprendizaje de 0.05 y 127 hojas. Su average precision es de 0.329 en validación y de 0.386 en prueba, contra 0.133 y 0.168 del baseline. Con solo 4.36 % de bloques en riesgo en prueba, la exactitud sería engañosa: Treveil et al. (2020) dan el ejemplo de una clase positiva de 5 %, donde un modelo que siempre predice la negativa acierta el 95 % y es inútil (p. 51). La average precision mide qué tan bien ordena el modelo los bloques por probabilidad de riesgo, que es justo la decisión del equipo de rebalanceo: visitar primero las estaciones con mayor probabilidad, hasta donde alcance la capacidad del camión. Con el umbral calibrado en validación (0.826), en prueba encuentra el 42.5 % de los bloques en riesgo con una precisión del 39.3 %. Su ventaja sobre el `hgb` por defecto es pequeña (0.329 contra 0.320 en validación y 0.386 contra 0.376 en prueba), pero se confirma en los dos conjuntos. Cuesta más entrenarlo (6.5 s contra 2.0 s del `hgb` por defecto), pero sigue siendo un costo bajo.

Hay un detalle con el umbral. Queda registrado como parámetro y métrica del run, pero el modelo registrado devuelve la clase con el umbral de 0.5 de `predict()`. Quien consuma el modelo debe aplicar el umbral calibrado sobre `predict_proba()`. La elección no cambia, porque la average precision no depende del umbral. El siguiente paso es guardar el umbral dentro del modelo, por ejemplo con `FixedThresholdClassifier` de scikit-learn.

P3 · Grupos. Elegimos `kmeans` con k = 3, con una silueta de 0.096 contra 0.017 de agrupar por

municipio, aunque `make registry` propuso `gmm` por tener la silueta más alta. Antes de aceptarlo revisamos la estabilidad: con $k = 3$ y diez semillas distintas, fuera de MLflow, `gmm` cambia mucho de una semilla a otra (Tabla 11). Un modelo cuya partición cambia tanto con la semilla no es confiable para planear, y Davies-Bouldin y Calinski-Harabasz también prefieren a `kmeans`. Además, sus grupos se pueden describir con el perfil horario y la ubicación, que es lo que necesita el área de planeación. En costo no hay diferencia que pese: los tres entrenan en menos de un segundo (de 0.04 a 0.75 s). La silueta sigue siendo baja, porque los patrones de uso son continuos. Probar otras variables o un número mayor de grupos queda como trabajo futuro.

Modelo	Silueta (media ± desv.)	Estaciones en el grupo más grande	Grupo más pequeño
`kmeans`	0.098 ± 0.001	Entre el 44 % y el 48 %	56 estaciones o más
`gmm`	0.264 ± 0.033	Entre el 80 % y el 96 %	Una sola estación en dos de las diez semillas

Tabla 11. *Estabilidad de los candidatos de P3 con k = 3 y diez semillas (0 a 9), sobre los mismos datos y el mismo espacio escalado. El run oficial usa la semilla 42, con una silueta de 0.096 para `kmeans`.*

6. Model Registry y ciclo de vida

6.1 Modelos registrados

`make registry` registra los dos mejores candidatos de cada pregunta con un nombre que dice el equipo, la pregunta y la tarea (Tabla 12). Cada nombre agrupa todas las versiones de esa pregunta, y cada versión nueva incrementa el número de versión (MLflow, s.f.-b).

Pregunta	Modelo registrado	Versión `@champion`	Versión `@challenger`
P1	`mibici-equipo03-p1-demanda`	v1: `hgb` con la mejor combinación (run `a0290f62`)	v2: `hgb` por defecto
P2	`mibici-equipo03-p2-riesgo`	v1: `hgb` con la mejor combinación (run `d99bd3b0`)	v2: `hgb` por defecto
P3	`mibici-equipo03-p3-grupos`	v2: `kmeans` con $k = 3$ (run `674c77fa`)	v1: `gmm`

Tabla 12. *Modelos registrados en la corrida oficial.*

Cada modelo registrado tiene una descripción con lo que predice, la métrica con que se elige y cómo se carga, y las etiquetas `question` y `metrica_seleccion`. Cada versión tiene una descripción con su métrica contra la del baseline y su origen, y las etiquetas de linaje de la Tabla 13.

Etiqueta	Contenido
`experiment`	`mibici-equipo03`
`source_run`	Run ID de la corrida que produjo el modelo
`git_commit`	Commit del código con que se entrenó (`ee51bc9...`)
`data_version`	Versión de los datos con que se entrenó (`4ab2182e7493`)
`motivo_promocion`	Solo en la versión de P3 promovida por el equipo: el motivo de la promoción

Tabla 13. *Etiquetas de linaje de cada versión registrada.*

El Registry ya liga cada versión con el run que la produjo (MLflow, s.f.-b). Las etiquetas repiten ese linaje junto con el commit y la versión de los datos para que se vea completo en la página del modelo, sin tener que abrir el run. Lauchande (2021) describe el Registry como un almacén central donde los modelos de una organización se comparten, se crean y se archivan de forma colaborativa (p. 24), y donde cada modelo registrado se anota con sus metadatos (p. 25).

6.2 Esquema de ciclo de vida: aliases en lugar de etapas

Usamos aliases: `champion` para la versión en producción y `challenger` para la siguiente mejor. No usamos las etapas clásicas (*Staging*, *Production* y *Archived*) por estas razones:

- **Las etapas están obsoletas.** Desde MLflow 2.9.0 están marcadas como obsoletas y se quitarán en una versión mayor futura, porque resultaron poco flexibles para expresar flujos de MLOps. Los aliases y las etiquetas de versión son su reemplazo (MLflow, s.f.-f). Usamos MLflow 3.16.1.
- **Separan el despliegue del código que consume el modelo.** Quien use el modelo lo carga con `models:/mibici-equipo03-p1-demanda@champion`. Si el alias se mueve a otra versión, la siguiente carga toma la nueva sin cambiar ese código (MLflow, s.f.-f).
- **Permiten comparar antes de promover y regresar.** Una versión puede tener varios aliases y un alias se puede reasignar. Eso facilita mantener un `challenger` listo para pruebas A/B o para reemplazar al `champion`, y volver a la versión anterior con solo mover el alias (MLflow, s.f.-b, s.f.-f).
- **El paso a producción solo ocurre después de las validaciones.** Treveil et al. (2020) proponen que un modelo no se despliegue hasta pasar todas sus validaciones (p. 37). `make registry` solo asigna `champion` a un candidato que le gana al baseline en validación y en prueba, con los mismos datos y la misma partición, desde un árbol sin cambios, y después comprueba que el modelo se carga por su nombre y alias y predice sobre su ejemplo de entrada. Si ningún candidato pasa, la pregunta queda sin `champion` y el comando termina con error.

Con las etapas, el modelo pasaría de *Staging* a *Production* con una transición (Lauchande, 2021, p. 25). Con los aliases, «producción» es la versión a la que apunta `champion`. Correr `make registry` otra vez no crea versiones repetidas, porque si el modelo de un run ya está registrado, reutiliza su versión y solo actualiza aliases, etiquetas y descripciones.

Promoción con revisión del equipo (P3). `make registry` aplica los criterios automáticos y propone el `champion`. Los criterios de estabilidad y costo los revisa el equipo. En P3 esa revisión cambió la propuesta: movimos `champion` a la versión de `kmeans` y dejamos `gmm` como `challenger`, con el motivo en la etiqueta `motivo_promocion` y en las descripciones de las dos versiones. Mover un alias es la operación que propone el flujo de trabajo de MLflow para promover una versión (MLflow, s.f.-f), y en código son dos llamadas:

```
from mlflow import MlflowClient

cliente = MlflowClient()
cliente.set_registered_model_alias("mibici-equipo03-p3-grupos", "champion", 2)    # kmeans
cliente.set_registered_model_alias("mibici-equipo03-p3-grupos", "challenger", 1)  # gmm
```

Si en el futuro se vuelve a correr `make registry`, propondrá otra vez `gmm`, porque su regla solo mira la silueta. Para que esa decisión no dependa de un paso manual, el siguiente paso es incorporar a `make registry` un criterio de estabilidad, por ejemplo un tamaño mínimo de grupo o la silueta con varias semillas.

6.3 Carga por nombre y alias

```
import mlflow

modelo = mlflow.pyfunc.load_model("models:/mibici-equipo03-p1-demanda@champion")
modelo.predict(modelo.input_example)
```

`make registry` hace esta misma carga con los tres modelos y predice sobre el ejemplo de entrada de cada uno. Su salida en la corrida oficial fue:

```
P1: mibici-equipo03-p1-demanda v1 @champion (val_mae = 5.828 contra 7.835
    del baseline, +25.6%); predice 3 filas de su ejemplo
P2: mibici-equipo03-p2-riesgo v1 @champion (val_average_precision = 0.329
    contra 0.133 del baseline, +148.1%); predice 3 filas de su ejemplo
P3: mibici-equipo03-p3-grupos v1 @champion (train_silhouette = 0.277 contra
    0.017 del baseline, +1498.0%); predice 3 filas de su ejemplo
```

Después del cambio de alias de P3, la misma carga de `models:/mibici-equipo03-p3-grupos@champion` devuelve la versión 2 (`kmeans`) y predice las 3 filas de su ejemplo. Las Figuras 7 a 10 muestran los tres modelos en producción con su linaje.

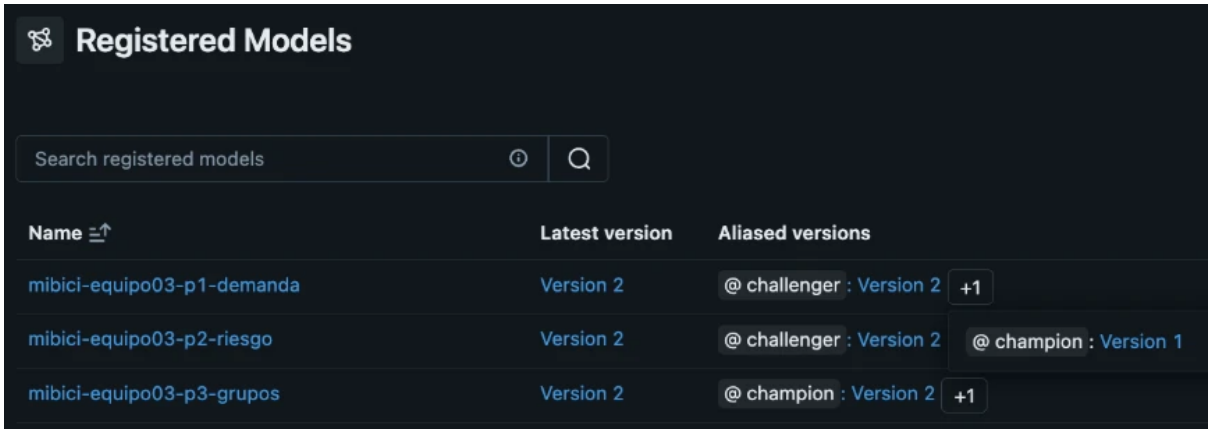


Figura 7. *Los tres modelos registrados. En P1 y P2, `@champion` apunta a la versión 1 (se ve al abrir «+1») y `@challenger` a la versión 2. En P3, `@champion` apunta a la versión 2 (`kmeans`).*

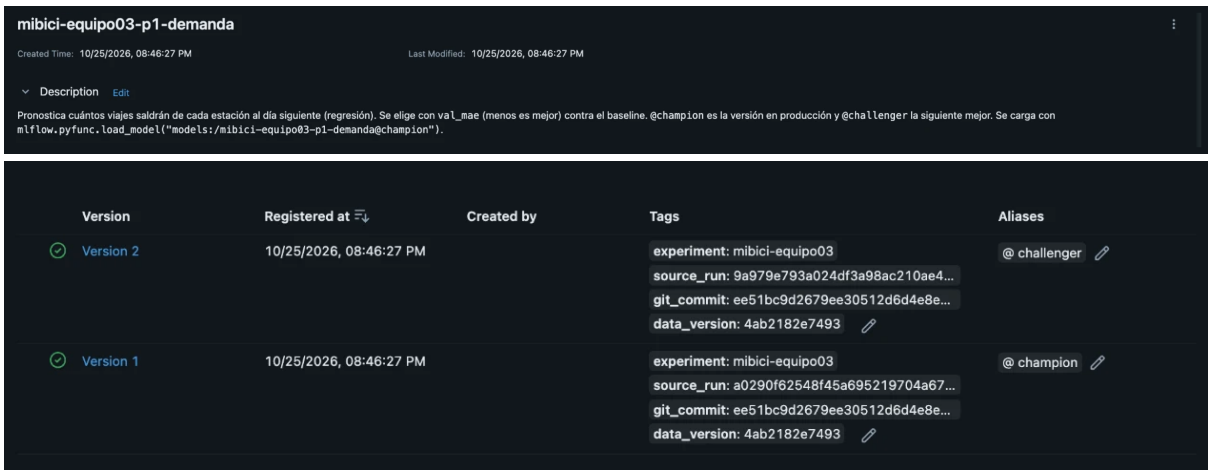


Figura 8. *`mibici-equipo03-p1-demanda`: arriba, la descripción del modelo registrado. Abajo, sus dos versiones con alias y etiquetas de linaje (`experiment`, `source_run`, `git_commit` y `data_version`).*

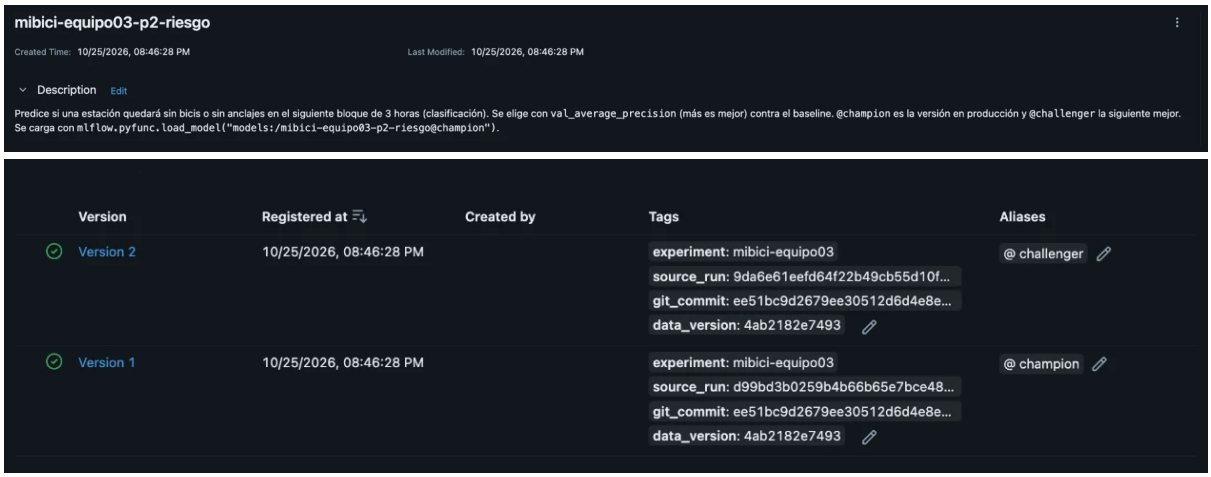


Figura 9. *`mibici-equipo03-p2-riesgo`: descripción y sus dos versiones, con la versión 1 en `@champion` y su linaje.*

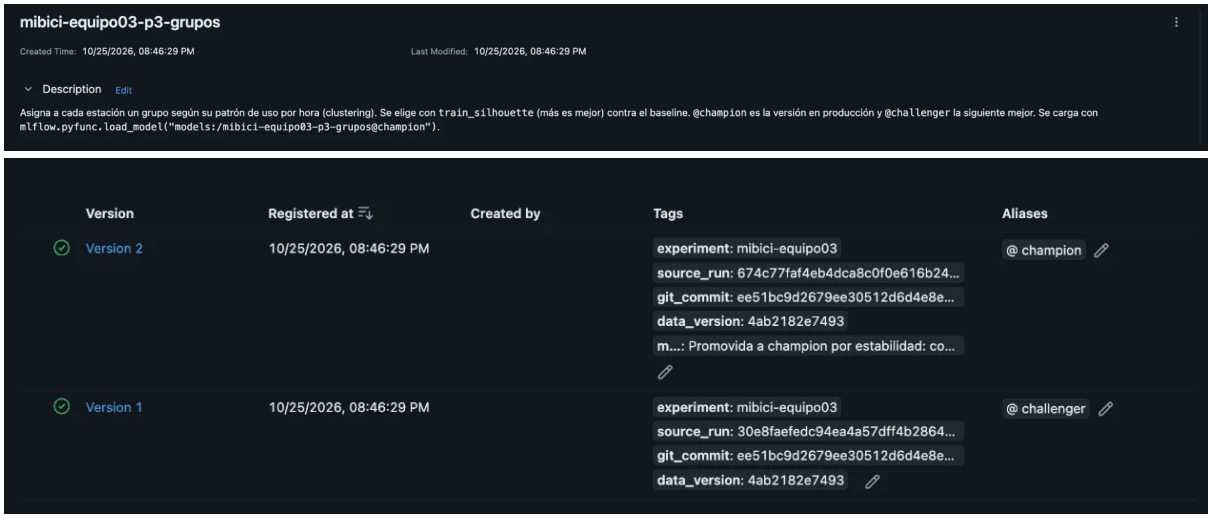


Figura 10. *`mibici-equipo03-p3-grupos`: descripción y sus dos versiones, con la versión 2 (`kmeans`) en `@champion`, la etiqueta `motivo_promocion` y la versión 1 (`gmm`) como `@challenger`.*

7. Flujo de trabajo en Git

Todo el trabajo de esta entrega entró a `main`, que está protegida, por Pull Request revisado por otro integrante, con ramas `tipo/descripcion`, Conventional Commits en español y `Closes #N` en la descripción. Además, desde el #59 cada Pull Request corre en GitHub Actions el lint y las pruebas. La Tabla 14 resume los Pull Requests de la entrega. Los cuatro integrantes abrieron al menos uno y revisaron al menos uno.

PR	Issue	Qué agrega	Abrió	Revisó
#48	#35	`python_env.yaml` y entrada `ingest`	Gerardo	Miguel
#49	#36	Entrada `train` con catálogo de modelos	Gerardo	Sebastián
#50	#37	Validación temporal y métrica de selección	Gerardo	Miguel
#52	#51	Un archivo por pregunta y columnas en `config.py`	Gerardo	Sebastián
#53	#43	Model Registry y `make registry`	Sebastián	Gerardo
#54	#38	Etiquetas y artefactos organizados	Gerardo	Sebastián
#55	#34	Versión de datos por contenido	Sebastián	Gerardo
#56	#39	Búsqueda con nested runs	Gerardo	Anaí
#57	#40	Candidatos de P1	Miguel	Anaí y Gerardo
#58	#33	Servidor de tracking con SQLite	Anaí	Gerardo
#59	—	Integración continua: lint y pruebas en cada Pull Request	Miguel	Gerardo
#60	#41	Candidatos de P2 y umbral calibrado	Miguel	Gerardo
#61	#42	Candidatos de P3	Miguel	Gerardo
#62	#44	README: `mlflow run`, búsqueda y aliases	Gerardo	Anaí
#63	#45	`make experimentos` y corrida oficial	Sebastián	Gerardo
#64	#45	README: `make experimentos`, modelos elegidos y champion de P3	Sebastián	Gerardo
#65	#46	Capturas de MLflow de esta entrega	Sebastián	Gerardo

Tabla 14. *Pull Requests de la segunda entrega.*

Este reporte entra a `main` con su propio Pull Request, que cierra el issue #47, con el PDF en `reports/`.

8. Conclusiones

El repositorio quedó como un MLflow Project que cualquier integrante reproduce desde un clon limpio con la ingesta, `make experimentos` y `make registry`. La corrida oficial desde `main` lo confirmó, con el mismo manifiesto, la misma versión de los datos y las mismas métricas que en los ensayos.

En las tres preguntas, el modelo elegido le gana a su baseline en validación y en prueba. En P1, `hgb` baja el error de 6.9 a 5.5 viajes por estación y día. En P2, `hgb` más que duplica la average precision (de 0.168 a 0.386) con más recall y la misma precisión. En P3, `kmeans` con tres grupos supera la agrupación por municipio con grupos estables y fáciles de describir.

Para elegir no nos quedamos con la mejor cifra. En P1 descartamos un modelo cien veces más pesado que no mejoraba el error, y en P3 la revisión de estabilidad cambió la propuesta automática. Cada modelo está en el Registry con su linaje completo y su versión en producción marcada con `@champion`, que es como lo consumirán las siguientes etapas del proyecto. Lo que sigue es servir los modelos por su alias, monitorear su desempeño con datos nuevos, reentrenar cuando los datos cambien, guardar el umbral de P2 dentro del modelo y llevar el criterio de estabilidad de P3 a `make registry`.

Referencias

Anthropic. (2026). *Claude* (versión Opus 5.5) [Modelo de lenguaje grande], utilizado mediante Claude Code para apoyo en la redacción y en la escritura y revisión de código. https://claude.com/claude-code

Lauchande, N. (2021). *Machine learning engineering with MLflow: Manage the end-to-end machine learning life cycle with MLflow*. Packt Publishing.

MLflow. (s.f.-a). *MLflow dataset tracking* [Documentación]. Recuperado el 25 de octubre de 2026, de https://mlflow.org/docs/latest/ml/dataset/

MLflow. (s.f.-b). *MLflow Model Registry* [Documentación]. Recuperado el 25 de octubre de 2026, de https://mlflow.org/docs/latest/ml/model-registry/

MLflow. (s.f.-c). *MLflow Projects* [Documentación]. Recuperado el 25 de octubre de 2026, de https://mlflow.org/docs/latest/ml/projects/

MLflow. (s.f.-d). *MLflow Tracking* [Documentación]. Recuperado el 25 de octubre de 2026, de https://mlflow.org/docs/latest/ml/tracking/

MLflow. (s.f.-e). *MLflow Tracking APIs* [Documentación]. Recuperado el 25 de octubre de 2026, de https://mlflow.org/docs/latest/ml/tracking/tracking-api/

MLflow. (s.f.-f). *Model Registry workflows* [Documentación]. Recuperado el 25 de octubre de 2026, de https://mlflow.org/docs/latest/ml/model-registry/workflow/

scikit-learn developers. (s.f.-a). *Clustering*. scikit-learn user guide. Recuperado el 17 de octubre de 2026, de https://scikit-learn.org/stable/modules/clustering.html

scikit-learn developers. (s.f.-b). *Metrics and scoring: Quantifying the quality of predictions*. scikit-learn user guide. Recuperado el 17 de octubre de 2026, de https://scikit-learn.org/stable/modules/model_evaluation.html

Treveil, M., Omont, N., Stenac, C., Lefevre, K., Phan, D., Zentici, J., Lavoillotte, A., Miyazaki, M., & Heidmann, L. (2020). *Introducing MLOps: How to scale machine learning in the enterprise*. O'Reilly Media.